



Seja bem-vindo a nossa atividade de seleção para a vaga de Desenvolvedor júnior (Full-Stack).

O objetivo deste teste é avaliar suas habilidades fundamentais de codificação, arquitetura de software, design de interfaces e resolução de problemas práticos. Este é um projeto completamente isolado, não será utilizado como ferramenta da empresa, e deve ser construído do zero por você.

### **O DESAFIO: "TASK TRACKER & ANALYTICS"**

Você deverá construir uma aplicação web simples para gerenciamento e análise de tarefas/atividades diárias. A aplicação deve contar com uma interface de usuário (Frontend) dinâmica e uma API (Backend) para processar e armazenar as informações.



#### **REQUISITOS DO FRONTEND**

##### **1. Interface do Usuário (UI/UX):**

- Uma página responsiva, limpa e moderna (sugerimos usar Tailwind CSS ou CSS moderno).
- Exibição das tarefas em formato de lista ou grid, contendo título, descrição, status (Pendente / Em Andamento / Concluída) e prioridade (Alta, Média, Baixa).
- Formulário para criação e edição de tarefas.

##### **2. Painel de Estatísticas (Analytics):**

- Um pequeno dashboard ou seção com resumos visuais das tarefas (ex: porcentagem de conclusão, quantidade de tarefas por nível de prioridade).
- Uso de componentes visuais agradáveis (gráficos simples com alguma biblioteca como Recharts, ou barras de progresso estilizadas).

##### **3. Estados de UI:**

- Tratamento visual para estado de carregamento (Loading state).
- Estado para lista vazia (Empty state).



## ⚙️ REQUISITOS DO BACKEND (API)

### 1. Endpoints de CRUD:

- GET /api/tasks - Retorna a lista de tarefas.
- POST /api/tasks - Cria uma nova tarefa (com validação de campos).
- PUT /api/tasks/:id - Atualiza o status, prioridade ou detalhes da tarefa.
- DELETE /api/tasks/:id - Remove uma tarefa.

### 2. Lógica de Agrupamento/Estatísticas:

- O backend deve expor um endpoint (ex: GET /api/tasks/stats) ou processar os dados para retornar estatísticas calculadas (ex: total de tarefas concluídas, percentual de progresso, distribuição por prioridade).

### 3. Persistência:

- Você pode utilizar o banco de dados de sua preferência (PostgreSQL, SQLite, MongoDB) ou, para simplificar, persistência em arquivo JSON local ou em memória (especifique a escolha no README).

## 🔧 TECNOLOGIAS RECOMENDADAS

Você tem total liberdade para escolher a stack com a qual se sente mais confortável, mas recomendamos tecnologias alinhadas ao nosso ecossistema atual:

- Linguagem: TypeScript (fortemente recomendada)
- Frontend: React (Next.js, Vite ou similar)
- Backend: Node.js (Next.js API Routes, Express, Fastify ou NestJS)

## 📦 O QUE VAMOS AVALIAR?

Ao analisar a sua entrega, focaremos nos seguintes pilares:

### 1. Qualidade do Código e Organização:

- Estrutura de pastas clara e intuitiva.



- Código legível, componentização eficiente e reaproveitamento de lógica.
- Uso correto de tipos em TypeScript (evitar uso excessivo de any).

## 2. Resolução de Problemas & Lógica:

- Como os dados são validados antes de salvar no backend.
- Como as estatísticas são calculadas e estruturadas na API.
- Tratamento de erros (ex: tentar atualizar uma tarefa que não existe).

## 3. Design e UX/UI:

- Uma interface limpa, consistente, com bom contraste de cores, espaçamentos e responsividade para dispositivos móveis.

## 4. Boas Práticas de Git:

- Commits organizados e fáceis de acompanhar.



## **INSTRUÇÕES DE ENTREGA**

1. Crie um repositório público no GitHub.
2. Escreva um arquivo README.md contendo:
  - Instruções passo a passo de como rodar o projeto localmente (frontend e backend).
  - Breve descrição da arquitetura adotada e das principais bibliotecas utilizadas.
  - Se utilizou algum banco de dados ou persistência em memória.
3. Envie o link do repositório para o responsável pela sua avaliação.
4. Prazo final para entrega dia 26/06/2026 às 12h.

Boa sorte! Estamos ansiosos para ver sua solução.